Lab 5: Static Code Analysis

Retail Point of Sale & Discount Tracker (pos_system.py)

Name: Thejaswi S **SRN:** PES2UG24CS562

GitHub repository: https://github.com/RockingThej/Retail_Pos

1. Summary of Results

The program was analysed with Pylint, Bandit and Flake8 before and after the fixes. All three tools now report clean results, and the program still performs its job.

Tool	Before	After
Pylint	5.65 / 10 (20 messages)	10.00 / 10 (0 messages)
Bandit	3 issues (1 Medium, 2 Low)	No issues identified
Flake8	12 warnings	0 warnings

2. Issue Documentation Table

Line numbers refer to the original file and were checked against the before reports.

Issue	Type	Line(s)	Description	Fix Approach
eval() on user input (Bandit B307, Medium; Pylint W0123)	Security	27	apply_promo_code runs whatever the user types, so input such as __import__("os").system(...) executes arbitrary code.	Removed eval. Promo codes are matched against a strict regex (total * N or total - N) and calculated manually; anything else raises ValueError.
Mutable default argument receipts=[] (Pylint W0102)	Bug	10	The list is created once and shared by every call, so state leaks between calls.	Removed the parameter. Totals are kept in self.receipts, created per PosSystem instance.
Bare except (Pylint W0702, Flake8 E722)	Bug / Style	16	Hides every error. The unknown item "laptop" and the string quantity "2" were silently billed as 0.	Replaced with explicit validation: unknown items and invalid quantities are skipped with a logged warning.
try/except/pass and broad except (Bandit B110, Low; Pylint W0718)	Security / Bug	33-34	update_stock swallows every error, and its only statement (price_list[item] = price_list[item]) does nothing.	Rewritten to update a real stock dict, warn on unknown items, and return True/False.
random.randint for receipt IDs (Bandit B311, Low)	Security / Bug	21	Pseudo-random numbers are not suitable for IDs and can collide (range 1000-9999).	Replaced with a sequential counter (itertools.count(1001)).
Negative quantity accepted	Logic bug (not flagged by any tool)	47	Bill 3 ("bottle", -4) produced a total of -630.00 and reduced sales_total.	Quantities must be positive integers; invalid lines are skipped with a warning.
Invalid tax rate accepted	Logic bug (not flagged by any tool)	48	tax_rate=-1 made the bill total exactly 0.	ValueError unless 0 <= tax_rate <= 1; main catches it and prints a message.
File never closed (Pylint R1732, W1514)	Resource leak	37	open() used without "with" and without an encoding.	with open(file, "a", encoding="utf-8") as handle:
Unused imports json, logging (Pylint W0611, Flake8 F401)	Code quality	1, 3	Dead imports.	Removed json; logging is now used for warnings.

Issue	Type	Line(s)	Description	Fix Approach
Unused variable total (Flake8 F841, Pylint W0612)	Code quality	26	total was only "used" inside the eval string, so analysis sees it as dead.	Disappeared once eval was removed.
Global state and global statement (Pylint W0603, C0103 on sales_total)	Design	7, 11	Module-level mutable globals with non-constant naming.	Moved all state into a PosSystem class.
Function name printSales (Pylint C0103)	Style	41	Not snake_case.	Renamed to print_sales.
Missing docstrings (Pylint C0114, C0116)	Style	Module, 10, 24, 29, 36, 41, 44	No module or function docstrings.	Added docstrings to the module, class and every method.
% string formatting (Pylint C0209)	Style	21	Old-style string formatting.	Converted to f-strings.
Line too long, 88 > 79 (Flake8 E501)	Style	21	The receipt append line exceeded 79 characters.	Built the receipt text in a short f-string on its own line.
Blank lines (Flake8 E302, E305)	Style	10, 24, 29, 36, 41, 44, 56	Two blank lines are required around top-level definitions.	Added two blank lines before the class, main and the entry-point guard.
Unguarded main() call	Design	56	The script runs when imported.	Added if __name__ == "__main__":.

3. Git Commits

The changes were committed in the following order, with the analysis reports kept alongside them:

- **Add before static analysis reports** (pylint, bandit and flake8 before reports)
- **Fix Bandit issues: remove eval, try/except/pass, random IDs** (bandit_report_after.txt)
- **Fix Pylint issues: class refactor, docstrings, validation, with-open** (pylint_report_after.txt, score 10.00/10)
- **Fix Flake8 style issues: blank lines** (flake8_report_after.txt, no warnings)

4. Reflection Questions

Question 1: Which issue found by the tools was the most serious, and what could it have caused?

The most serious issue was the use of eval() in apply_promo_code (Bandit B307, Medium severity, also flagged by Pylint as W0123). The function passed whatever the cashier typed at the promo prompt straight into eval(), so a value such as __import__("os").system(...) would run as real code on the machine. In a point-of-sale system this could mean deleted files, stolen sales data or tampered totals. Even a harmless input such as __import__("os").getcwd() would be executed and return the current folder, which shows that the input is treated as code.

I fixed it by removing eval completely. Promo codes are now checked against a strict regular expression that only accepts the two documented forms, "total * N" and "total - N", and the discount is calculated with normal arithmetic. Anything else raises a ValueError that main() reports as "Invalid promo code". Bandit went from 3 issues to none.

Question 2: Which bugs did the static analysis tools not report, and how did you find them?

Several real logic errors are invisible to Pylint, Bandit and Flake8, because the code is syntactically valid. I found them by running the program and reading the output carefully. Bill 2 silently ignored the unknown item "laptop". Bill 3 produced a negative total of -630.00, which also reduced the running sales total. Bill 4

returned 0, because tax_rate=-1 cancels the subtotal and the string quantity "2" caused a TypeError that the bare except swallowed. update_stock also did nothing useful: it reassigned price_list[item] to itself, and printed nothing for unknown items.

The tools did point at the cause indirectly. The bare except (W0702/E722) and the except-pass (B110) were the code that was hiding these failures. I fixed the logic by validating items, quantities and the tax rate explicitly. This shows that static analysis finds risky patterns but cannot know the intended behaviour, so running the program and testing remain necessary.

Question 3: How did restructuring the code into a PosSystem class improve its quality and maintainability?

The original code kept sales_total, price_list and receipt_lines as module-level globals and used the global statement (Pylint W0603 and an invalid-name warning). It also had a mutable default argument, receipts=[], that was shared between calls. Moving this state into a PosSystem class means each instance owns its own prices, stock, totals and receipts. Nothing leaks between calls, and the code can be tested by creating a fresh PosSystem for each test.

The class also made the responsibilities clearer: process_checkout validates and totals a cart, apply_promo_code only calculates a discount, update_stock manages stock, and save_receipt_log only writes the file. I also added docstrings, replaced % formatting with f-strings, used with open(..., encoding="utf-8") so the log file is always closed, and renamed printSales to print_sales. Together these changes raised the Pylint score from 5.65 to 10.00.

Question 4: How do Pylint, Bandit and Flake8 differ, and how would you use them in a real development workflow?

The three tools overlap but have different strengths. Bandit looks only for security problems (eval, try/except/pass, weak random numbers). Flake8 checks PEP 8 style and simple errors (blank lines, line length, unused imports and variables). Pylint is the broadest: it reviews design and quality, such as docstrings, naming, global state, mutable defaults and resource handling, and gives an overall score. Some problems were reported by more than one tool, for example the bare except (Pylint W0702 and Flake8 E722) and unused imports (Pylint W0611 and Flake8 F401), while only Bandit reported the security findings.

In a real project I would run all three automatically, before each commit (for example with pre-commit hooks) and in a CI pipeline, so that new code cannot be merged with a failing check. I would prioritise Bandit findings and Pylint errors first, then warnings, and treat style issues last. I would also remember that a clean report is not proof of correct behaviour, so the tools should be combined with unit tests and code review.

5. Final Code: pos_system.py

```
"""Retail point-of-sale and discount tracker."""

import logging
import re
from itertools import count

logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
LOGGER = logging.getLogger(__name__)

DEFAULT_PRICES = {"pen": 10, "notebook": 45, "bag": 799, "bottle": 150}

# Accepts only codes such as "total * 0.9" or "total - 50".
PROMO_PATTERN = re.compile(r"^total\s*([*-])\s*(\d+(?:\.\d+)?)$")


class PosSystem:
    """Tracks prices, stock, sales totals and receipts for a small shop."""

    def __init__(self, prices=None):
        self.price_list = dict(prices or DEFAULT_PRICES)
        self.stock = {}
        self.sales_total = 0.0
        self.receipts = []
        self.receipt_lines = []
        self._receipt_numbers = count(1001)

    def process_checkout(self, cart_items, tax_rate=0.05):
        """Total a cart including tax and record the receipt."""
        if not 0 <= tax_rate <= 1:
            raise ValueError(f"Invalid tax rate: {tax_rate}")
        subtotal = 0
        for name, qty in cart_items:
            if name not in self.price_list:
                LOGGER.warning("Unknown item skipped: %s", name)
            elif not isinstance(qty, int) or qty <= 0:
                LOGGER.warning("Invalid quantity for %s: %r", name, qty)
            else:
                subtotal += self.price_list[name] * qty
        total = subtotal * (1 + tax_rate)
        self.receipts.append(total)
        self.sales_total += total
        number = next(self._receipt_numbers)
        self.receipt_lines.append(f"Receipt #{number} total={total:.2f}")
        return total

    def apply_promo_code(self, code_str):
        """Apply a promo code like 'total * 0.9' to the running total."""
        match = PROMO_PATTERN.match(code_str.strip())
        if match is None:
            raise ValueError(f"Unsupported promo code: {code_str!r}")
        operator, value = match.group(1), float(match.group(2))
        if operator == "*" and value > 1:
            raise ValueError("Promo multiplier cannot exceed 1")
        if operator == "*":
            result = self.sales_total * value
        else:
            result = self.sales_total - value
        return max(result, 0.0)
```

```
    def update_stock(self, item, qty):
        """Add qty units of a known item to stock."""
        if item not in self.price_list:
            LOGGER.warning("Cannot update stock, unknown item: %s", item)
            return False
        self.stock[item] = self.stock.get(item, 0) + qty
        LOGGER.info("Stock updated for %s by %d", item, qty)
        return True

    def save_receipt_log(self, file="pos_log.txt"):
        """Append all receipt lines to a log file."""
        with open(file, "a", encoding="utf-8") as handle:
            for line in self.receipt_lines:
                handle.write(line + "\n")

    def print_sales(self):
        """Print the running sales total."""
        print(f"Sales so far: {self.sales_total:.2f}")


def main():
    """Run a demo checkout session."""
    pos = PosSystem()
    print("Bill 1:", pos.process_checkout([("pen", 3), ("notebook", 2)]))
    print("Bill 2:", pos.process_checkout([("bag", 1), ("laptop", 1)]))
    print("Bill 3:", pos.process_checkout([("bottle", -4)]))
    try:
        print("Bill 4:", pos.process_checkout([("pen", "2")], tax_rate=-1))
    except ValueError as error:
        print("Bill 4 rejected:", error)
    pos.update_stock("laptop", 5)
    print("After promo:", pos.apply_promo_code("total * 0.9"))
    code = input("Enter promo code (e.g. total - 50): ") or "total - 50"
    try:
        print("After your promo:", pos.apply_promo_code(code))
    except ValueError as error:
        print("Invalid promo code:", error)
    pos.save_receipt_log()
    pos.print_sales()


if __name__ == "__main__":
    main()
```